



Domain Driven Design - Java

Prof. Gilberto Alexandre das Neves
profgilberto.neves@fiap.com.br

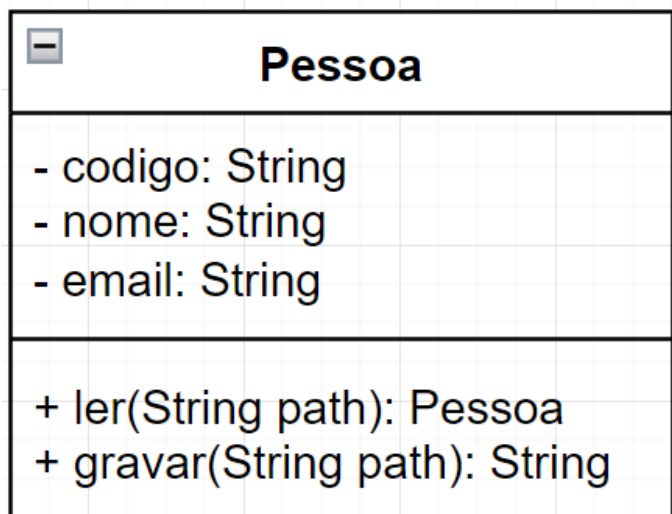
Manipulação de Arquivos

Existem diversas classes que podem ser usadas para realizar o armazenamento e a recuperação de um arquivo texto. Vamos utilizar a classe **BufferedReader** para a leitura do arquivo e a classe **PrintWriter** para a gravação.

Implemente a classe **Pessoa** no pacote **bean** conforme modelo UML.

Orientações:

- Definir apenas construtor vazio.
- Definir todos os *getters* e *setters*.
- Método **ler()** lê o arquivo da pasta (*path*) informado e retorna um objeto da classe **Pessoa** (delegue o tratamento deste erro à classe **IOException**).
- Método **gravar()** salva o arquivo na pasta (*path*) informado e retorna mensagem de erro ou falha.
- **Obs.:** nome do arquivo deve ser formado pelo **código** e extensão **.txt**



```
public class Pessoa {  
    private String codigo;  
    private String nome;  
    private String email;  
  
    public Pessoa() {  
    }  
  
    public String getCodigo() {  
        return codigo;  
    }  
  
    public void setCodigo(String codigo) {  
        this.codigo = codigo;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

```
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public Pessoa ler(String path) throws IOException {  
    BufferedReader br = new BufferedReader(  
        new FileReader(path + "/" + codigo + ".txt"));  
    codigo = br.readLine();  
    nome = br.readLine();  
    email = br.readLine();  
    br.close();  
    return this;  
}
```

```
public String gravar(String path) {  
    try {  
        File dir = new File(path);  
        if (!dir.exists()) {  
            dir.mkdir();  
        }  
        PrintWriter pw = new PrintWriter(path + "/" + codigo + ".txt");  
        pw.println(codigo);  
        pw.println(nome);  
        pw.println(email);  
        pw.flush();  
        pw.close();  
        return "Arquivo gravado com sucesso";  
    } catch (IOException e) {  
        return "Falha ao gravar arquivo: " + e.getMessage();  
    }  
}
```

Implemente a classe **UsaPessoa** (pacote **main**) com o método **main**. A classe deve utilizar a classe **JOptionPane** para interagir com o usuário, veja a seguir as orientações:

1. Exibir um menu perguntando para o usuário que operação ele deseja executar (cadastrar ou consultar uma pessoa).
2. Peça para o usuário informar o caminho da pasta e instancie o objeto da classe **Pessoa**.
3. Execute a operação escolhida (peça as informações necessárias para cada operação e exiba resultados).
4. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 4 novamente).
5. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.

```
public class UsaPessoa {  
    public static void main(String[] args) {  
        String aux, codigo, nome, email, path;  
        int opcao;  
        Pessoa pessoa;  
        do {  
            try {  
                aux = JOptionPane.showInputDialog("Escolha:\n1.Cadastrar\n2.Consultar");  
                opcao = Integer.parseInt(aux);  
                path = JOptionPane.showInputDialog("Digite caminho da pasta");  
                pessoa = new Pessoa();  
                switch (opcao) {  
                    case 1:  
                        codigo = JOptionPane.showInputDialog("Digite o codigo");  
                        nome = JOptionPane.showInputDialog("Digite o nome");  
                        email = JOptionPane.showInputDialog("Digite o e-mail");  
                        pessoa.setCodigo(codigo);  
                        pessoa.setNome(nome);  
                        pessoa.setEmail(email);  
                        JOptionPane.showMessageDialog(null, pessoa.gravar(path));  
                        break;  
                }  
            }  
        }  
    }  
}
```



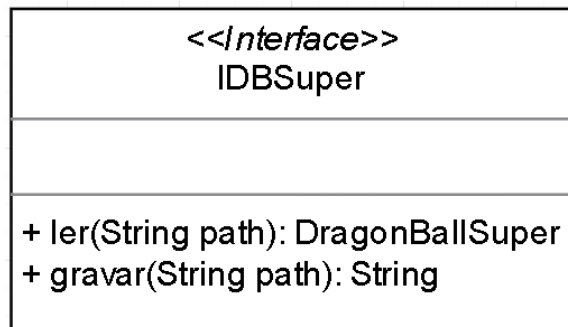
```
case 2:
    codigo = JOptionPane.showInputDialog("Digite o codigo");
    pessoa.setCodigo(codigo);
    pessoa = pessoa.ler(path);
    if (pessoa == null) {
        JOptionPane.showMessageDialog(null,
            "Caminho e/ou codigo informado inexistente");
    } else {
        JOptionPane.showMessageDialog(null, "Exibindo dados:"
            + "\nCaminho: " + path
            + "\nArquivo: " + path + "/" + pessoa.getCodigo() + ".txt"
            + "\nCodigo: " + pessoa.getCodigo()
            + "\nNome: " + pessoa.getNome()
            + "\nE-mail: " + pessoa.getEmail());
    }
    break;
default:
    JOptionPane.showMessageDialog(null, "Escolha incorreta");
}
```

```
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(null, "Erro de Conversão!\n" + e.getMessage(),
            "ERRO", JOptionPane.ERROR_MESSAGE);
    } catch (IOException e) {
        JOptionPane.showMessageDialog(null, "Erro ao Acessar Arquivo!\n" + e.getMessage
            (), "ERRO", JOptionPane.ERROR_MESSAGE);
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null, "Erro: " + e.getMessage(), "ERRO",
            JOptionPane.ERROR_MESSAGE);
    }
} while (JOptionPane.showConfirmDialog(null, "Deseja continuar?", "Atenção",
    JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE) == 0);
JOptionPane.showMessageDialog(null, "Fim de programa");
}
```

Praticando...

Classe DragonBallSuper

Implemente a classe **DragonBallSuper** no pacote **bean** conforme orientações:



Orientações:

- Criar a *Interface IDBSuper*
- Criar modelo **UML** da classe (usar o *Card* como referência).
- Criar a classe **DragonBallSuper** implementando a *Interface*.
- Definir apenas construtor vazio.
- Definir todos os *getters* e *setters*.
- Método **ler()** lê o arquivo da pasta (*path*) informado e retorna um objeto da classe **DragonBallSuper** (delegue o tratamento deste erro à classe **IOException**).
- Método **gravar()** salva o arquivo na pasta (*path*) informado e retorna mensagem de erro ou falha.
- Incluir **Javadoc** na classe e métodos.
- **Obs.:** nome do arquivo deve ser formado pelo **nome_do_personagem** e extensão **.txt**

Implemente a classe **UsaDBSuper** (pacote **main**) com o método **main**. A classe deve utilizar a classe **JOptionPane** para interagir com o usuário, veja a seguir as orientações:

1. Exibir um menu perguntando para o usuário que operação ele deseja executar (cadastrar ou consultar um personagem de Dragon Ball Super).
2. Peça para o usuário informar o caminho da pasta e instancie o objeto da classe **DragonBallSuper**.
3. Execute a operação escolhida (peça as informações necessárias para cada operação e exiba resultados).
4. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 4 novamente).
5. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.

1. Qual método da classe `BufferedReader` é comumente utilizado para ler uma linha de texto de um arquivo?
 - a) `read()`
 - b) `readLine()`
 - c) `readText()`
 - d) `readAll()`

2. Qual é a principal função do método `println()` da classe `PrintWriter`?
 - a) Escrever uma linha de texto no console
 - b) Escrever uma string de texto em um array
 - c) Ler uma linha de texto de um arquivo
 - d) Escrever uma linha de texto em um arquivo, seguido por um caractere de nova linha



Java como programar. Paul Deitel e Harvey Deitel. Pearson, 2011.

Java 8 – Ensino Didático : Desenvolvimento e Implementação de Aplicações. Sérgio Furgeri. Editora Érica, 2015.

Até breve!

1. Qual método da classe `BufferedReader` é comumente utilizado para ler uma linha de texto de um arquivo?
 - a) `read()`
 - ☒ b) `readLine()`
 - c) `readText()`
 - d) `readAll()`

2. Qual é a principal função do método `println()` da classe `PrintWriter`?
 - a) Escrever uma linha de texto no console
 - b) Escrever uma string de texto em um array
 - c) Ler uma linha de texto de um arquivo
 - ☒ d) Escrever uma linha de texto em um arquivo, seguido por um caractere de nova linha